

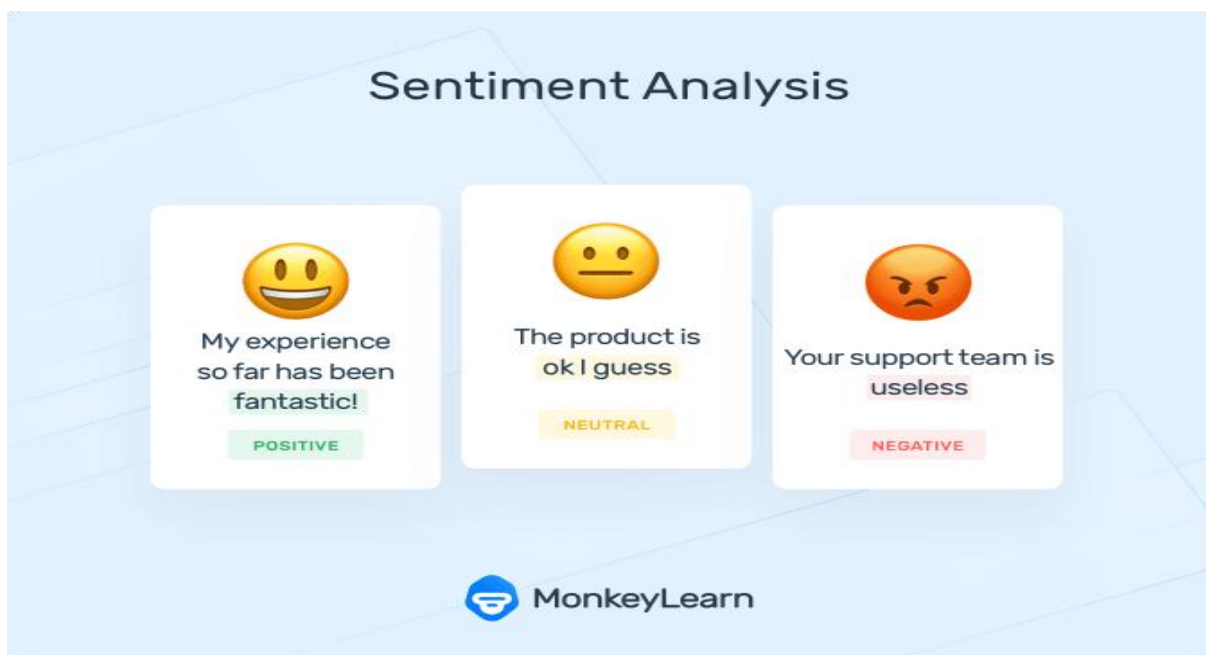
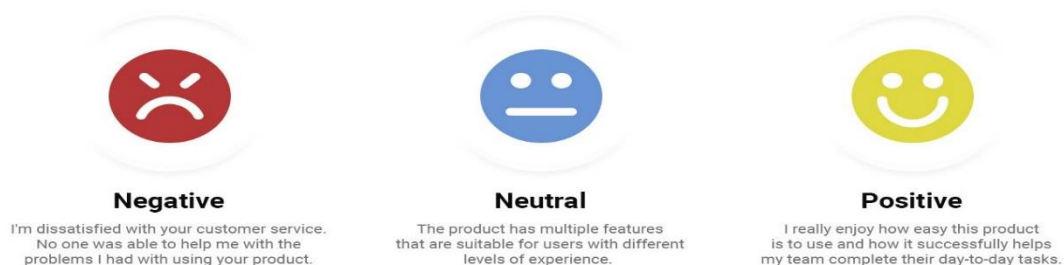
Sentiment Analysis (SA)

What is Sentiment analysis

Sentiment analysis is a natural language processing (NLP) technique used to determine whether data is positive, negative, or neutral. Sentiment analysis is often performed on textual data to help businesses monitor brand and product sentiment in customer feedback, and understanding customer needs.

Sentiment analysis is contextual mining of words which indicates the social sentiment of a brand and also helps the business to determine whether the product which they are manufacturing is going to make a demand in the market or not. The goal which Sentiment analysis tries to gain is to analyze people's opinion in a way that it can help the businesses expand. It focuses not only on polarity (positive, negative & neutral) but also on emotions (happy, sad, angry, etc.).

Sentiment Analysis



Why is Sentiment Analysis Important in Business?

- 1- You can aggregate customer sentiment from free-form feedback data and identify the primary promoters or detractors of your product or service. Once the detractors have been converted to promoters, you can gradually implement corrective measures to rebuild trust.
- 2- It is possible to detect hateful and inappropriate comments on your online platform, which will help you keep it free of bullies.
- 3- The feedback you receive from recent employees, peer reviews, and manager feedback can help you identify demotivated or about-to-quit-employees and provide them with constructive ways to succeed.

Based on these sentiment analysis examples, you can see that sentiment analysis is an excellent tool for gaining a better understanding of employees and customers, keeping platforms safe, improving customer shopping, and learning from competitors.

Applications of Sentiment Analysis

1- Social Media Monitoring

Social media posts often contain some of the most honest opinions about your products, services, and businesses because they're unsolicited. By incorporating NLP experts on board, you can analyze individual emotions and overall public sentiment across all social platforms based on their comments, feedback, and reviews.

In chat, sentiment analysis can detect sarcasm, understand acronyms (OMG, LOL, ROFL, etc.), correct misspellings and misuses, and detect sarcasm.

2- Managing & Monitoring Business Reputation

Sentiment analysis is widely used in business for brand monitoring.

Through which, you can monitor your progress over time by keeping track of your brand's image and reputation. You can use this information to develop usable statistics and information about your brand.

3- Analyzing the Market Response to a Product

Learn what users are saying about your new product soon after launch, or examine feedback you may have yet to see. Identify what makes your most valuable customers happy, learn how your product is perceived by your target audience, and improve your product. The whole thing is based on sentiment analysis.

4- Get to Know the Voice of Your Customers (VoCs)

Creating a personalized customer experience requires communication and listening to the voice of your customers. As a result, businesses have access to a large number of conversation sentiment analysis datasets that can be integrated into the development of ML models in order to automate the sentiment analysis process. Taking all feedback, you receive from customers, such as web feedback, surveys, chats, and customer service calls, and evaluating it. This data can be categorized and structured with the help of sentiment analysis so that you can discover recurring topics and concerns.

5- Analyzing Market & Competitors

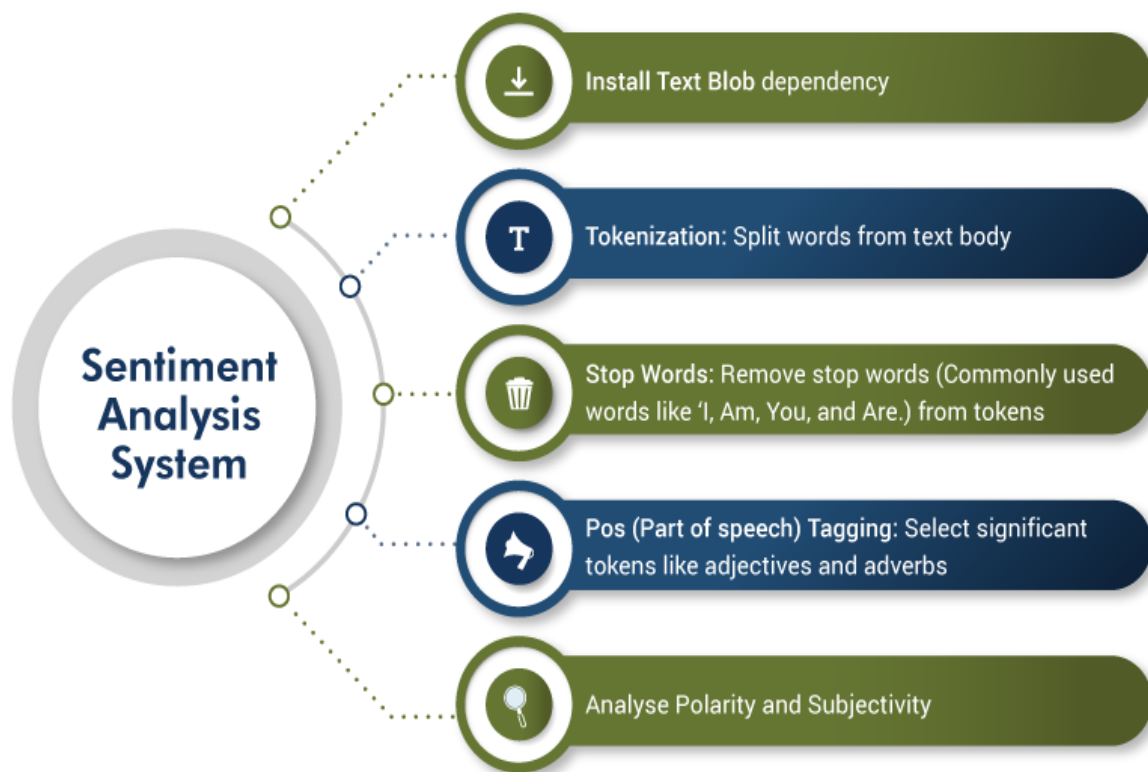
In order to research the market and your competitors, you can use sentiment analysis. Get a sense of who your competitors are mentioning positively and how your marketing efforts compare. Look at how your competitors speak positively to their customers, and take some of those phrases and use them in your own brand message and tone of voice guide.

How does Sentiment analysis works

Sentiment analysis follows a basic number of rules, as follows:

1. First, it breaks down a text document (or phrase) into certain categories (i.e., by sentence, phrase, and so forth).
2. It identifies every 'sentiment' phrase.
3. It scores each sentiment phrase from -1 to +1.

The screenshot displays a sentiment analysis tool interface. At the top, there is a red header bar with the text "Results" and a "Legend" link. Below the header, a text document is shown with several phrases highlighted in colored boxes and numbered 1 through 6. The text reads: "I really enjoyed using the Canon Ixus in Madrid on March 4. The Panasonic Lumix is a bit disappointing, but the Canon camera is not bad at all. All I want when taking photos is point it and then just press the button. For only 200 dollars, a really fair price, this camera is perfect for me. Besides, I have had a good customer service experience." The highlighted phrases are: "really enjoyed using" (1, green), "Canon Ixus" (1, green), "Panasonic Lumix" (2, red), "is a bit disappointing" (2, red), "Canon camera" (3, green), "is not bad" (3, green), "really fair" (4, green), "price" (4, green), "camera" (5, green), "perfect" (5, green), "good" (6, green), and "customer service experience" (6, green).



Types of Sentiment Analysis

The sentiment analysis process mainly focuses on polarity, i.e., positive, negative, or neutral. Apart from polarity, it also considers the feelings and emotions(happy, sad, angry, etc.), intentions(interested or not interested), or urgency(urgent or not urgent) of the text.

1. Fine-grained Sentiment Analysis

If your business requires the polarity precisions, then you can classify your polarity categories into the following parts:

- Very positive
- Positive
- Neutral
- Negative
- Very Negative

For polarity analysis, you can use the 5-star ratings as a customer review where very positive refers to a five-star rating and very negative refers to a one-star rating.

2. Emotion Detection

This type of sentiment analysis helps to detect customer emotions like happiness, disappointment, anger, sadness, etc. Here, you can use sentiment lexicons or complex machine learning algorithms to identify the customer's feelings.

One of the disadvantages of using sentiment lexicons is that people tend to express emotions in different ways. So, it may be confusing to understand human emotion clearly while using it.

3. Aspect-based Sentiment Analysis

Let's say that you are analyzing customer sentiment using fine-grained analysis. You want to identify the particular aspect or features for which people are mentioning positive or negative reviews. Here, aspect-based sentiment analysis comes into play.

For instance, in the review "The camera quality of this phone is getting worse with time," an aspect-based classifier will determine that the review expresses a negative opinion from the customer for the phone's camera feature.

4. Multilingual Sentiment Analysis

Multilingual sentiment analysis is complex compared to others as it includes many preprocessing and resources available online (i.e., sentiment lexicons). Businesses value the feedback of the customer regardless of their geography or language. Therefore, multilingual sentiment analysis helps you identify customer sentiment irrespective of location or language difference.

TextBlob

TextBlob is a **Lexicon-based sentiment analyzer**. It has some predefined rules or we can say word and weight dictionary, where it has some scores that help to calculate a sentence's polarity. That's why the Lexicon-based sentiment analyzers are also called "Rule-based sentiment analyzers".

Sentiment analysis using TextBlob. The TextBlob's sentiment property returns a **Sentiment object**. The polarity indicates sentiment with a value from -1.0 (negative) to 1.0 (positive) with 0.0 being neutral. The subjectivity is a value from 0.0 (objective) to 1.0 (subjective).

NaiveBayesClassifier

It is a **classification technique based on Bayes' Theorem with an assumption of independence among predictors.**

Tokenize Method

Transform input tensors of strings into output tokens

What is Keras

What Is Keras? It is a open source neural network library. Keras is a high-level, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. It also supports multiple backend neural network computation.

TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks

Sentiment Analysis using textblob and NaiveBayesClassifier

```
from textblob import TextBlob
```

```
from textblob.classifiers import NaiveBayesClassifier
```

```
train = [
```

```
    ('I love this sandwich.', 'pos'),
```

```
    ('This is an amazing place!', 'pos'),
```

```
    ('I feel very good about these beers.', 'pos'),
```

```
    ('I do not like this restaurant', 'neg'),
```

```
    ('I am tired of this stuff.', 'neg'),
```

```
    ("I can't deal with this", 'neg'),
```

```
    ("My boss is horrible.", "neg")
```

```
]
```

```
cl = NaiveBayesClassifier(train)
```

```
print("The polarity of sentence I feel amazing is",cl.classify("I feel amazing!"))
```

```
blob = TextBlob("The beer is good. But the hangover is horrible. I can't drive", classifier=cl)
```

```
for s in blob.sentences:
```

```
    print(s)
```

```
print(s.classify())
```

OUTPUT

The polarity of sentence I feel amazing is pos

The beer is good.

pos

But the hangover is horrible.

neg

I can't drive

neg

Sentiment Analysis using textblob

```
from textblob import TextBlob
```

```
feedbacks = ['I love the app is amazing ',
```

```
            "The experience was bad as hell",
```

```
            "This app is really helpful",
```

```
            "Damn the app tastes like shit ",
```

```
            'Please don\'t download the app you will regret it ']
```

```
positive_feedbacks = []
```

```
negative_feedbacks = []
```

```
for feedback in feedbacks:
```

```
    feedback_polarity = TextBlob(feedback).sentiment.polarity
```

```
    if feedback_polarity > 0:
```

```
        positive_feedbacks.append(feedback)
```

```
        continue
```

```
    negative_feedbacks.append(feedback)
```

```
print('Positive_feedbacks Count : {}'.format(len(positive_feedbacks)))
```

```
print(positive_feedbacks)
```

```
print('Negative_feedback Count : {}'.format(len(negative_feedbacks)))
```

```
print(negative_feedbacks)
```

OUTPUT

Positive_feedbacks Count : 2

['I love the app is amazing ', 'This app is really helpful']

Negative_feedback Count : 3

['The experience was bad as hell', 'Damn the app tastes like shit ', "Please don't download the app you will regret it "]

Sentiment Analysis using tensorflow keras

```
import pandas as pd
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, SpatialDropout1D
from tensorflow.keras.layers import Embedding
df = pd.read_csv("C:/Users/Hp/OneDrive/Desktop/Datasets/Tweets.csv")
df.head()
```

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	airline	airline_sentiment_gold	name
0	570306133677760513	neutral	1.0000	NaN	NaN	Virgin America	NaN	cairdin
1	570301130888122368	positive	0.3486	NaN	0.0000	Virgin America	NaN	jnardino
2	570301083672813571	neutral	0.6837	NaN	NaN	Virgin America	NaN	yvonnalynn
3	570301031407624196	negative	1.0000	Bad Flight	0.7033	Virgin America	NaN	jnardino
4	570300817074462722	negative	1.0000	Can't Tell	1.0000	Virgin America	NaN	jnardino

```
tweet_df = df[['text', 'airline_sentiment']]
tweet_df = tweet_df[tweet_df['airline_sentiment'] != 'neutral']
print(tweet_df.shape)
tweet_df.head(5)
```

```
(11541, 2)
```

	text	airline_sentiment
1	@VirginAmerica plus you've added commercials t...	positive
3	@VirginAmerica it's really aggressive to blast...	negative
4	@VirginAmerica and it's a really big bad thing...	negative
5	@VirginAmerica seriously would pay \$30 a fligh...	negative
6	@VirginAmerica yes, nearly every time I fly VX...	positive

```
tweet_df["airline_sentiment"].value_counts()
```

```
negative    9178
```

```
positive    2363
```

```
Name: airline_sentiment, dtype: int64
```

```
sentiment_label = tweet_df.airline_sentiment.factorize()
```

```
sentiment_label
```

```
(array([0, 1, 1, ..., 0, 1, 1], dtype=int64),
```

```
Index(['positive', 'negative'], dtype='object'))
```

```
tweet = tweet_df.text.values
```

```
tokenizer = Tokenizer(num_words=5000)
```

```
tokenizer.fit_on_texts(tweet)
```

```
vocab_size = len(tokenizer.word_index) + 1
```

```
encoded_docs = tokenizer.texts_to_sequences(tweet)
```

```
padded_sequence = pad_sequences(encoded_docs, maxlen=200)
```

```
print(tokenizer.word_index)
```

```
{'to': 1, 'the': 2, 'i': 3, 'a': 4, 'united': 5, 'you': 6, 'for': 7, 'flight': 8, 'and': 9, 'on': 10, 'my': 11, 'usairways': 12, 'americanair': 13, 'is': 14, 'in': 15, 'southwestair': 16, 'of': 17, 'jetblue': 18, 'me': 19, 'your': 20, 'it': 21, 'was': 22, 'not': 23, 'no': 24, 'have': 25, 'at': 26, 'with': 27, 'that': 28, 'this': 29, 'get': 30, 'but': 31, 'be': 32, 'canceled': 33, 'thanks': 34, 'now': 35, 'service': 36, 'are': 37, 'we': 38, 'from': 39, 'an': 40, 'been': 41, 'just': 42, '2': 43, 'so': 44, 'customer': 45, 'help': 46, 't': 47, 'can': 48, 'time': 49, 'co': 50, 'up': 51, 'hours': 52, 'http': 53, 'do': 54, 'hold': 55, 'they': 56, 'out': 57, 'amp': 58, 'plane': 59, 'i'm': 60, 'us': 61, 'all': 62, 'will': 63, 'why': 64, 'thank': 65, 'still': 66, 'our': 67, 'delayed': 68, 'what': 69, 'when': 70, 'how': 71, 'one': 72, 'can't': 73, 'flights': 74, 'call': 75, 'gate': 76, 'hour': 77, 'had': 78, 'flightled': 79, 'back': 80, 'bag': 81, 'if': 82, 'would': 83, 'after': 84, 'has': 85, 'about': 86, 'there': 87, 'it's': 88, 'don't': 89, 'as': 90, 'got': 91, 'late': 92, 'phone': 93, 'need': 94, 'please': 95, 'again': 96, '3': 97, 'airline': 98, 'over': 99, 'or': 100, 'like': 101, 'waiting': 102, 'virginamerica': 103, 'today': 104, 'more': 105, 'am': 106, 'guys': 107, 'by': 108, '4': 109, 'great': 110, 'wait': 111, 'u': 112, 'because': 113, 'fly': 114, 'never': 115, 'day': 116, 'trying': 117, 'i've': 118, 'airport': 119, 'then': 120, 'delay': 121, 'only': 122, '1': 123, 'really': 124, 'minu
```

```
print(tweet[0])
```

```
print(encoded_docs[0])
```

```
@VirginAmerica plus you've added commercials to the experience... tacky.
```

```
[103, 575, 530, 1287, 2416, 1, 2, 177]
print(padded_sequence[0])
embedding_vector_length = 32
model = Sequential()
model.add(Embedding(vocab_size, embedding_vector_length, input_length=200))
model.add(SpatialDropout1D(0.25))
model.add(LSTM(50, dropout=0.5, recurrent_dropout=0.5))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
print(model.summary())
history = model.fit(padded_sequence, sentiment_label[0], validation_split=0.2, epochs=5, batch_size=32)
```

```
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 2416 1 2 177]
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 32)	423488
spatial_dropout1d (SpatialDropout1D)	(None, 200, 32)	0
lstm (LSTM)	(None, 50)	16600
dropout (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

```
=====
Total params: 440,139
Trainable params: 440,139
Non-trainable params: 0
```

```
None
Epoch 1/5
289/289 [=====] - 52s 164ms/step - loss: 0.4149 - accuracy: 0.8244 - val_loss: 0.2168 - val_accuracy: 0.9225
Epoch 2/5
289/289 [=====] - 47s 161ms/step - loss: 0.2239 - accuracy: 0.9149 - val_loss: 0.1673 - val_accuracy:
```

```
def predict_sentiment(text):
    tw = tokenizer.texts_to_sequences([text])
    tw = pad_sequences(tw,maxlen=200)
    prediction = int(model.predict(tw).round().item())
    print("Predicted label: ", sentiment_label[1][prediction])
```

```
test_sentence1 = "I enjoyed my journey on this flight."
```

```
predict_sentiment(test_sentence1)
```

```
test_sentence2 = "This is the worst flight experience of my life!"
```

```
predict_sentiment(test_sentence2)
```

```
1/1 [=====] - 0s 374ms/step
Predicted label:  positive
1/1 [=====] - 0s 47ms/step
Predicted label:  negative
```

SENTIMENTAL ANALYSIS OF MOVIE REVIEWS USING KERAS AND TENSORFLOW

```
import pandas as pd
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
df=pd.read_csv("/Test.csv")
```

```
df.head()
```

	text	label
0	I always wrote this series off as being a comp...	0
1	1st watched 12/7/2002 - 3 out of 10(Dir-Steve ...	0
2	This movie was so poorly written and directed ...	0
3	The most interesting thing about Miryang (Secr...	1
4	when i first read about "berlin am meer" i did...	0

```
df["label"].value_counts()
```

```
1    2505
0    2495
Name: label, dtype: int64
```

```
sentiment_label = df.label.factorize()
sentiment_label
```

```
(array([0, 0, 0, ..., 0, 0, 0]), Int64Index([0, 1], dtype='int64'))
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM,Dense,Dropout,SpatialDropout1D
from tensorflow.keras.layers import Embedding
```

```
tweet = df.text.values
tokenizer = Tokenizer(num_words=5000)
tokenizer.fit_on_texts(tweet)
vocab_size = len(tokenizer.word_index) + 1
encoded_docs = tokenizer.texts_to_sequences(tweet)
padded_sequence = pad_sequences(encoded_docs, maxlen=200)
embedding_vector_length = 32
```

```
model = Sequential()
model.add(Embedding(vocab_size, embedding_vector_length, input_length=200) )
model.add(SpatialDropout1D(0.25))
model.add(LSTM(50, dropout=0.5, recurrent_dropout=0.5))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy',optimizer='adam', metrics=['accuracy'])
```

```
print(model.summary())
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 32)	1395264
spatial_dropout1d (SpatialD ropout1D)	(None, 200, 32)	0
lstm (LSTM)	(None, 50)	16600
dropout (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

```
=====
Total params: 1,411,915
Trainable params: 1,411,915
Non-trainable params: 0
```

```
history = model.fit(padded_sequence,sentiment_label[0],validation_split=0.2, epochs=5, batch_size=32)
```

```
Epoch 1/5
125/125 [=====] - 27s 197ms/step - loss: 0.6671 - accuracy: 0.5903 - val_loss: 0.5840 - val_accuracy: 0.7340
Epoch 2/5
125/125 [=====] - 24s 191ms/step - loss: 0.4681 - accuracy: 0.7915 - val_loss: 0.4236 - val_accuracy: 0.8150
Epoch 3/5
125/125 [=====] - 24s 196ms/step - loss: 0.3263 - accuracy: 0.8660 - val_loss: 0.4047 - val_accuracy: 0.8160
Epoch 4/5
125/125 [=====] - 25s 203ms/step - loss: 0.2486 - accuracy: 0.9075 - val_loss: 0.4049 - val_accuracy: 0.8430
Epoch 5/5
125/125 [=====] - 24s 190ms/step - loss: 0.2620 - accuracy: 0.8940 - val_loss: 0.4420 - val_accuracy: 0.8450
```

```
def predict_sentiment(text):
    tw = tokenizer.texts_to_sequences([text])
    tw = pad_sequences(tw,maxlen=200)
    prediction = int(model.predict(tw).round().item())
    rev=sentiment_label[1][prediction]
    if rev==0:
        print("Negative")
    else:
        print("Positive")
```

```
t1="This movie was really bad"
predict_sentiment(t1)
```

```
1/1 [=====] - 0s 35ms/step
Negative
```

```
t2="This is a funny film and I like it a lot"  
predict_sentiment(t2)
```

```
1/1 [=====] - 0s 41ms/step  
Positive
```

```
t3="I really enjoyed the detail that went into the script"  
predict_sentiment(t3)
```

```
1/1 [=====] - 0s 33ms/step  
Positive
```

Containers

A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.

Containers are running instance of the images

Container will package all your applications code and its dependencies and ship it to either public or private repositories and then download it and run it anywhere.

What is Docker??

Docker is a tool designed to **create, deploy, and run containers and container-based applications**. It can run on Linux, Windows, and macOS. Dockers allow developers to package applications into containers that combine application source code with all the dependencies and libraries needed to run the code in any environment.

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly.



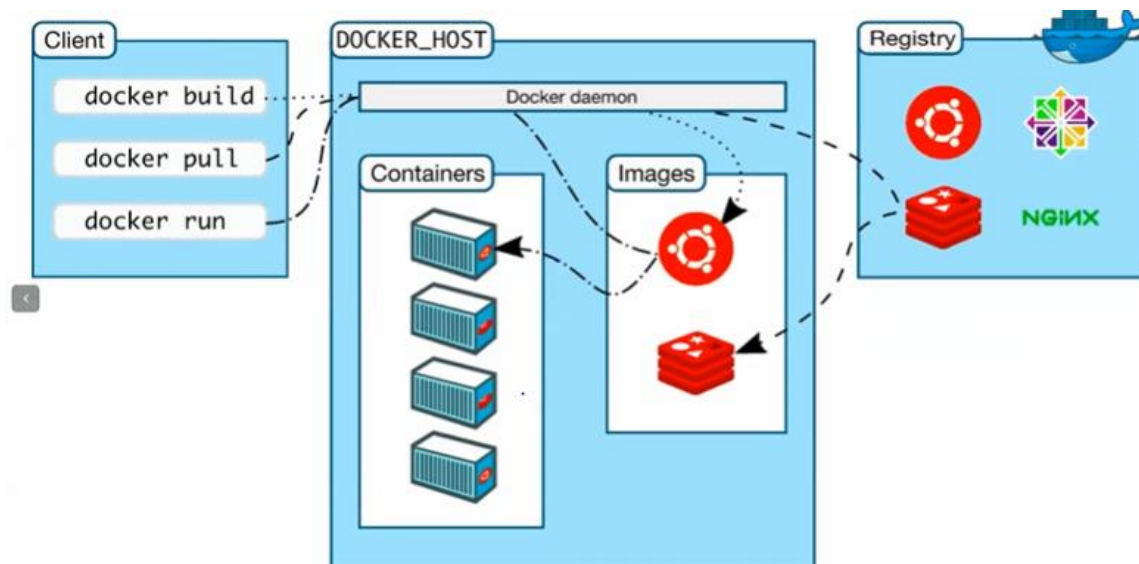
Advantages Of Docker

- Caching a cluster of containers
- Portability
- Images are light-weight
- Flexible resource sharing
- Scalability - many containers can be placed in a single host

- Running your service on hardware that is much cheaper than standard servers
- Fast deployment, ease of creating new instances, and faster migrations.
- Ease of moving and maintaining your applications
- Better security, less access needed to work with the code running inside containers, and fewer software dependencies

How does Dockers work?

Architecture of Docker



Docker works on a client-server architecture. It includes the docker client, docker host, and docker registry. The docker client is used for triggering docker commands, the docker host is used to running the docker daemon, and the docker registry to store docker images.

The docker client communicates to the docker daemon using a REST API, which internally supports to build, run, and distribution of docker containers. Both the client and daemon can run on the same system or can be connected remotely.

- We use the client (CLI) to issue a build command to the Docker daemon for building a docker image. Based on our inputs the Docker daemon will build an image and save it in Registry, which can either be a local repository or Docker hub.
- If you don't want to create an image, just pull it from the docker hub built by another user.
- Finally, If we need to create a running instance of a docker image, issue a run command from CLI to create a docker container.

Components of a docker architecture.

Docker daemon: The docker daemon monitors API request and control docker objects like containers, images, volumes, and networks. For managing docker services, a daemon can also communicate with other daemons.

Docker Engine: It is the core part of the whole Docker system. Docker Engine is an application which follows **client-server architecture**. It is installed on the host machine. There are three components in the Docker Engine:

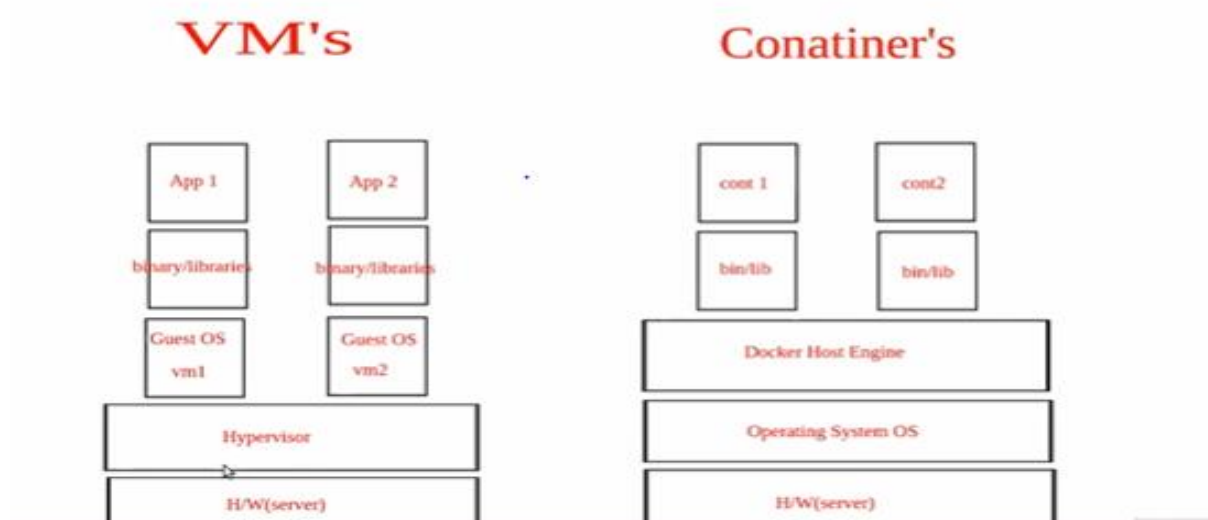
- **Server:** It is the docker daemon called **dockerd**. It can create and manage docker images, containers, networks, etc.
- **Rest API:** It is used to instruct docker daemon what to do.
- **Command Line Interface (CLI):** It is a client which is used to enter docker commands.

Docker Client: Docker users can interact with Docker through a client. When any docker commands runs, the client sends them to dockerd daemon, which carries them out. Docker API is used by Docker commands. Docker client can communicate with more than one daemon.

Docker Registries: It is the location where the Docker images are stored. It can be a public docker registry or a private docker registry. Docker Hub is the default place of docker images, its stores' public registry. You can also create and run your own private registry.

Docker objects : Docker images, containers, networks, volumes, plugins, etc. are the docker objects.

Difference Between VM and Docker



Virtual Machine	Dockers
Need Hypervisor	No Hypervisor required
Need to install package, libraries etc	No need ,because docker image can be pushed to repository
It takes a few minutes for VMs to boot.	Boots in a few seconds.
Requires entire OS to be loaded before starting the surface, so less efficient.	No space is needed to virtualize, hence less memory.

Docker Commands

Go to ec2 console and create ubuntu 18.04 instance

Change to root user by applying `sudo -i` or `sudo su`

`apt-get update`

`wget -qo- https://get.docker.com/ | sh`

`apt install docker.io -y`

Mobaxterm app can be used to perform docker commands.

`docker version`

`docker --help`

`docker info`

`docker images`

(Image is a template for the environment that you want to run and when you run a image you get a container)

Images:

Images are package or template such as VM Template

Images are used to create one or more containers

Images are read only template used to create containers

Images are stored in the Docker registry (Docker Hub)

Display / list out Docker Container

docker ps or (docker container ls)

docker ps -a or (docker container ls -a)

-a : indicating all containers such as active and in-active

Creating Container

i -> input

t -> terminal

--name : Make your own name of a container

Create a container with Attach Mode

docker run -it --name ourclock jpetazzo/clock

Ensure that Docker has created container or not

docker ps or docker container ls

docker ps -a or docker container ls -a

Create a container with background (De-attached mode)

docker run -d --name ourclock jpetazzo/clock

-d -> Deattached mode

You get the error as container exists on same name

docker ps

docker ps -a

Delete the container

```
docker rm < Container Id >
```

(Deleting Container)

Delete all container by single command

```
docker rm -f $(docker ps -a -q)
```

```
docker rmi < Image Id >
```

(Deleting Image)

Delete all images with single command

```
docker rmi $(docker images -qa)
```

Download images from Dockerhub

Port Forwarding

```
docker pull tutum/hello-world
```

```
docker run -it -p 80:80 --name web tutum/hello-world
```

-p is Port

(Run on the background)

Go to browser and check whether you get the output or not

Example : <http://ec2-54-161-80-134.compute-1.amazonaws.com/>

```
docker images
```

Now, you can upload your custom images to Docker Hub

```
docker login
```

(Docker connect to Docker Hub)

```
docker push <Docker Images>
```

Go to Docker Hub Repoistory where you can get image which you uploaded

Stop and Kill the Container

`docker stop < Container Id >`

Take few more seconds to stop the container

`docker kill < Container Id >`

Immediately kill the container

Deployment Strategies of DevOps

There are a variety of techniques to deploy new applications to production so choosing the right strategy is an important decision that needs to be made to leverage the impact of change on the consumer.

- **recreate:** version A is terminated then version B is rolled out
- **ramped (also known as rolling-update or incremental):** version B is slowly rolled out and replacing version A
- **blue/green:** version B is released alongside version A, then the traffic is switched to version B
- **canary:** version B is released to a subset of users, then proceed to a full rollout
- **a/b testing:** version B is released to a subset of users under specific condition
- **shadow:** version B receives real world traffic alongside version A and doesn't impact the response

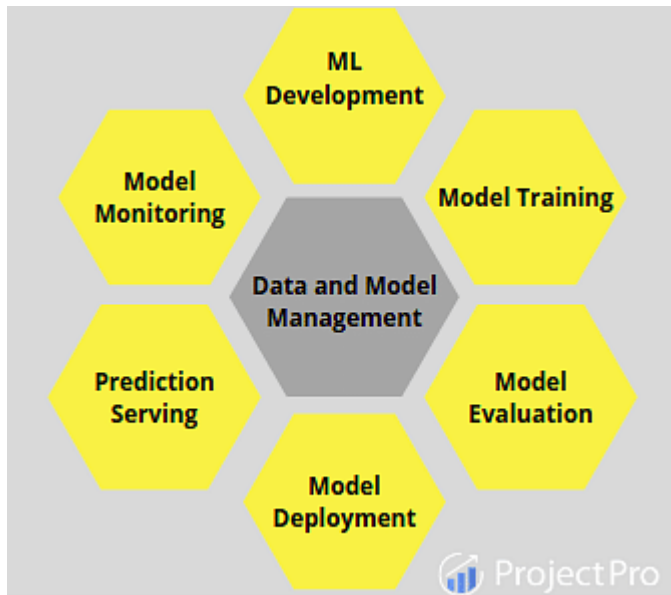
The most common strategy is to use a **blue-green deployment**. The new version (the blue version) is brought up for testing and evaluation, while the users still use the stable version (the green version). When ready, the users are switched to the blue version.

MLOps (Machine Learning Operations) in the Cloud

What is MLOps?

MLOps, also known as Machine Learning Operations for Production, is a set of standardized practices that can be utilized to build, deploy, and govern the lifecycle of ML models.

MLOps Lifecycle



Cloud Services for MLOps

MLOps can be hosted both on-premises and in the cloud:

Cloud-based MLOps provides access to a variety of managed services and features. Leading cloud providers let you run MLOps processes in the cloud, providing the tools and computing power you need, without having to procure and set up hardware and build an in-house ML environment.

Here are examples of services provided by the leading cloud providers:

- **Amazon SageMaker** is an ML platform that helps you build, train, manage, and deploy machine learning models in production-ready ML environments. SageMaker accelerates experiments with specialized tools for labeling, data preparation, training, tuning, and administrative monitoring.
- **Azure ML** is a cloud-based platform for training, deploying, automating, managing, and monitoring any machine learning experiment. Like SageMaker, it supports supervised and unsupervised learning.
- **Google Cloud** is an end-to-end, fully managed platform for machine learning and data science. There are features to help you manage ML services and create efficient ML workflows for developers, scientists, and data engineers. The platform enables fully automated machine learning lifecycle management.